

pohsun__40183452

October 15, 2025

#EECS 442 PS5: Backpropagation

Please make a copy of this file before editing! File > Save a copy in Drive

Please provide the following information (e.g. Andrew Owens, ahowens):

Po-Hsun Chang, pohsun

Important: after you download the .ipynb file, please name it ____<your_uniqname>__<your_umid>.ipynb__ before you submit it to Canvas.

Example: ahowens__00000000.ipynb.

1 Starting

Run the following code to import the modules you'll need. After your finish the assignment, remember to run all cells and save the note book to your local machine as a .ipynb file for Canvas submission.

```
[ ]: import pickle
import numpy as np
import matplotlib.pyplot as plt
import os
import math
from torchvision.datasets import CIFAR10
download = not os.path.isdir('cifar-10-batches-py')
dset_train = CIFAR10(root='.', download=download)
```

2 5.1 (b)

Implement the code for forward and backward pass of computation graph in (a)

```
[ ]: def f_1(x0, x1, x2, w0, w1, w2, w3):
    """
    Computes the forward and backward pass through the computational graph
    of (a)

    Inputs:
    - x0, x1, w0, w1, w2: Python floats
```

```

Returns a tuple of:
- L: The output of the graph
- grads: A tuple (grad_x0, grad_x1, grad_x2, grad_w0, grad_w1, grad_w2,
↳grad_w2)
giving the derivative of the output L with respect to each input.
"""

#####
# TODO: Implement the forward pass for the computational graph for (a) and#
# store the output of this graph as L                                     #
#####

a = w0 * x0
b = 1 / x1
c = w1 * b
d = w2 * x2
e = a + c
f = e - d
g = f + w3
h = -1 * g
i = np.exp(h)
j = 1 + i
L = 1 / j

#####
#                               END OF YOUR CODE                               #
#####

#####
# TODO: Implement the backward pass for the computational graph for (a) #
# Store the gradients for each input                                     #
#####

grad_L = 1.
↳0
↳dL_dL = 1
    grad_j = grad_L * -1 / (j ** 2)                                     #_
↳dL_dj = dL_dL * dL_dj
    grad_i = grad_j *_
↳1
↳dL_di = dL_dj * dj_di
    grad_h = grad_i * np.
↳exp(h)                                                                # dL_dh =_
↳dL_di * di_dh
    grad_g = grad_h *_
↳-1
↳dL_dg = dL_dh * dh_dg

```

```

    grad_f = grad_g *␣
↪1
↪dL_df = dL_dg * dg_df
    grad_w3 = grad_g *␣
↪1
↪dL_dw3 = dL_dg * dg_dw3
    grad_e = grad_f *␣
↪1
↪dL_de = dL_df * df_de
    grad_d = grad_f *␣
↪-1
↪dL_dd = dL_df * df_dd
    grad_a = grad_e *␣
↪1
↪dL_da = dL_de * de_da
    grad_c = grad_e *␣
↪1
↪dL_dc = dL_de * de_dc
    grad_w2 = grad_d *␣
↪x2
↪dL_dw2 = dL_dd * dd_dw2
    grad_x2 = grad_d *␣
↪w2
↪dL_dx2 = dL_dd * dd_dx2
    grad_w1 = grad_c *␣
↪b
↪dL_dw1 = dL_dc * dc_dw1
    grad_b = grad_c *␣
↪w1
↪dL_db = dL_dc * dc_db
    grad_x1 = grad_b * (-1 / (x1 ** 2)) # dL_dx1 = dL_db␣
↪* db_dx1
    grad_w0 = grad_a *␣
↪x0
↪dL_dw0 = dL_da * da_dw0
    grad_x0 = grad_a *␣
↪w0
↪dL_dx0 = dL_da * da_dx0

#####
#                               END OF YOUR CODE                               #
#####

grads = (grad_x0, grad_x1, grad_x2, grad_w0, grad_w1, grad_w2, grad_w3)
return L, grads

```

3 5.1 (c)

Implement the code for forward and backward pass of computation graph in (c)

```
[ ]: def f_2(x, y, z, w):
    """
    Computes the forward and backward pass through the computational graph
    of (c)

    Inputs:
    - x, y, z: Python floats

    Returns a tuple of:
    - L: The output of the graph
    - grads: A tuple (grad_x, grad_y, grad_z, grad_w)
    giving the derivative of the output L with respect to each input.
    """
    #####
    # TODO: Implement the forward pass for the computational graph for (c) and#
    # store the output of this graph as L                                     #
    #####

    a = 1 / w
    b = -1 * x
    c = np.exp(y)
    d = np.exp(z)
    e = a**b
    p = c / d
    f = c * p
    g = 1 / p
    m = e - f
    n = m + g
    L = 1 / n

    #####
    #                                     END OF YOUR CODE                                     #
    #####

    #####
    # TODO: Implement the backward pass for the computational graph for (c) #
    # Store the gradients for each input                                     #
    #####

    grad_L = 1.
    ↪0
    ↪dL_dL = 1
    grad_n = grad_L * -1 / (n ** 2)
    ↪dL_dn = dL_dL * dL_dn
```

```

    grad_g = grad_n * _
↪1
↪dL_dg = dL_dn * dn_dg
    grad_m = grad_n * _
↪1
↪dL_dm = dL_dn * dn_dm
    grad_e = grad_m * _
↪1
↪dL_de = dL_dm * dm_de
    grad_f = grad_m * _
↪-1
↪dL_df = dL_dm * dm_df
    grad_p_g = grad_g * -1 / (p ** 2) # dL_dp_g_
↪= dL_dg * dg_dp
    grad_c_f = grad_f * _
↪p
↪dL_dc_f = dL_df * df_dc
    grad_p_f = grad_f * _
↪c
↪dL_dp_f = dL_df * df_dp
    grad_p = grad_p_f + _
↪grad_p_g # dL_dp = dL_dp_f + _
↪dL_dp_g
    grad_c_p = grad_p * (1 / _
↪d) # dL_dc_p = dL_dp_
↪* dp_dc
    grad_c = grad_c_f + _
↪grad_c_p # dL_dc = dL_dc_f + _
↪dL_dc_p
    grad_d = grad_p * (-c / (d ** 2)) # dL_dd = _
↪dL_dp * dp_dd
    grad_b = grad_e * (a ** b) * np.log(a) # dL_db = dL_de * de_db
    grad_a = grad_e * (a ** b) * (b / a) # dL_da = dL_de * de_da
    grad_z = grad_d * np.
↪exp(z) # dL_dz = _
↪dL_dd * dd_dz
    grad_y = grad_c * np.
↪exp(y) # dL_dy = _
↪dL_dc * dc_dy
    grad_x = grad_b * _
↪-1
↪dL_dx = dL_db * db_dx
    grad_w = grad_a * -1 / (w ** 2) # _
↪dL_dw = dL_da * da_dw

```

```
#####
```

```

#                                     END OF YOUR CODE                                     #
#####

grads = (grad_x, grad_y, grad_z, grad_w)
return L, grads

```

4 Problem 5.2 Softmax Classifier with Two Layer Neural Network

In this problem you will develop a two Layer neural network with fully-connected layers to perform classification, and test it out on the CIFAR-10 dataset.

We train the network with a softmax loss function on the weight matrices. The network uses a ReLU nonlinearity after the first fully connected layer. In other words, the network has the following architecture:

input - fully connected layer - ReLU - fully connected layer - softmax

The outputs of the second fully-connected layer are the scores for each class.

You cannot use any deep learning libraries such as PyTorch in this part.

5 5.2 (a) Layers

In this problem, implement fully connected layer, relu. Softmax layer has already been implemented in the provided code. Filling in all TODOs in skeleton codes will be sufficient.

```

[ ]: def fc_forward(X, W, b):
    """
    Computes the forward pass for a fully-connected layer.

    The input X has shape (N, Din) and contains a minibatch of N
    examples, where each example x[i] has shape (Din,).

    Inputs:
    - X: A numpy array containing input data, of shape (N, Din)
    - W: A numpy array of weights, of shape (Din, Dout)
    - b: A numpy array of biases, of shape (Dout,)

    Returns a tuple of:
    - out: output, of shape (N, Dout)
    - cache: (X, W, b)
    """
    #####
    # TODO: Implement the forward pass. Store the result in out.      #
    #####
    out = X @ W + b
    #####
    #                                     END OF YOUR CODE                                     #

```

```

#####
cache = (X, W, b)
return out, cache

def fc_backward(dout, cache, L2_REG, lamda=0.01):
    """
    Computes the backward pass for a fully_connected layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, Dout)
    - cache: returned by your forward function. Tuple of:
      - X: Input data, of shape (N, Din)
      - W: Weights, of shape (Din, Dout)
      - b: Biases, of shape (Dout,)

    Returns a tuple of:
    - dX: Gradient with respect to X, of shape (N, Din)
    - dW: Gradient with respect to W, of shape (Din, Dout)
    - db: Gradient with respect to b, of shape (Dout,)
    """
    X, W, b = cache
    dX, dW, db = None, None, None
    #####
    # TODO: Implement the affine backward pass. #
    #####
    dX = dout @ W.T
    dW = X.T @ dout
    db = np.sum(dout, axis=0)
    if(L2_REG):
        dW += lamda * W
    #####
    #                               END OF YOUR CODE #
    #####
    return dX, dW, db

def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """

```

```

out = x.copy()
#####
# TODO: Implement the ReLU forward pass. #
#####
out[out < 0] = 0
#####
#                                     END OF YOUR CODE #
#####
cache = x
return out, cache

def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLU).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: returned by your forward function. Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = dout.copy(), cache
    #####
    # TODO: Implement the ReLU backward pass. #
    #####
    dx[x <= 0] = 0
    #####
    #                                     END OF YOUR CODE #
    #####
    return dx

def softmax_loss(X, y, L2_REG, W1, W2, lamda=0.01):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - X: Input data, of shape (N, C) where x[i, j] is the score for the jth
      class for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for X[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dX: Gradient of the loss with respect to x
    """

```



```

"""
loss, dX = None, None
dX = np.exp(X - np.max(X, axis=1, keepdims=True))
dX /= np.sum(dX, axis=1, keepdims=True)
loss = -np.sum(np.log(dX[np.arange(X.shape[0]), y])) / X.shape[0]
dX[np.arange(X.shape[0]), y] -= 1
dX /= X.shape[0]

if(L2_REG):
    loss += lamda * (np.sum(W1 * W1) + np.sum(W2 * W2))

return loss, dX

```

6 5.2 (b) Softmax Classifier

In this problem, implement softmax classifier.

```

[ ]: class SoftmaxClassifier(object):
    """
    A fully-connected neural network with
    softmax loss that uses a modular layer design. We assume an input dimension
    of D, a hidden dimension of H, and perform classification over C classes.

    The architecture should be fc - relu - fc - softmax with one hidden layer

    The learnable parameters of the model are stored in the dictionary
    self.params that maps parameter names to numpy arrays.
    """

    def __init__(self, input_dim=3072, hidden_dim=300, num_classes=10,
                  weight_scale=1e-3):
        """
        Initialize a new network.

        Inputs:
        - input_dim: An integer giving the size of the input
        - hidden_dim: An integer giving the size of the hidden layer, None
          if there's no hidden layer.
        - num_classes: An integer giving the number of classes to classify
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        """
        self.params = {}

        □
    ↪ #####

```

```

    # TODO: Initialize the weights and biases of the two-layer net. Weights
↪ #
    # should be initialized from a Gaussian centered at 0.0 with
↪ #
    # standard deviation equal to weight_scale, and biases should be
↪ #
    # initialized to zero. All weights and biases should be stored in the
↪ #
    # dictionary self.params, with fc weights and biases using the keys
↪ #
    # 'W' and 'b', i.e., W1, b1 for the weights and bias in the first
↪ linear #
    # layer, W2, b2 for the weights and bias in the second linear layer.
↪ #
    #
    #####
    self.params['W1'] = np.random.normal(0, weight_scale, (input_dim,
↪ hidden_dim))
    self.params['b1'] = np.zeros(hidden_dim)
    self.params['W2'] = np.random.normal(0, weight_scale, (hidden_dim,
↪ num_classes))
    self.params['b2'] = np.zeros(num_classes)
    #
    #####
    #                                     END OF YOUR CODE
↪ #
    #
    #####

def forwards_backwards(self, X, y=None, L2_REG=False):
    """
    Compute loss and gradient for a minibatch of data.

    Inputs:
    - X: Array of input data of shape (N, Din)
    - y: Array of labels, of shape (N,). y[i] gives the label for X[i].

    Returns:
    If y is None, then run a test-time forward pass of the model and return:
    - scores: Array of shape (N, C) giving classification scores, where
      scores[i, c] is the classification score for X[i] and class c.

    If y is not None, then run a training-time forward and backward pass.
↪ And
    return a tuple of:

```

```

- loss: Scalar value giving the loss
- grads: Dictionary with the same keys as self.params, mapping parameter
names to gradients of the loss with respect to those parameters.
"""
scores = None

↳ #####
# TODO: Implement the forward pass for the two-layer net, computing the
↳ #
# class scores for X and storing them in the scores variable.
↳ #

↳ #####
W1, b1 = self.params['W1'], self.params['b1']
W2, b2 = self.params['W2'], self.params['b2']
out1, cache1 = fc_forward(X, W1, b1)
out2, cache2 = relu_forward(out1)
scores, cache3 = fc_forward(out2, W2, b2)

↳ #####
#
#
↳ #

↳ #####
# If y is None then we are in test mode so just return scores
if y is None:
    return scores

loss, grads = 0, {}

↳ #####
# TODO: Implement the backward pass for the two-layer net. Store the
↳ loss #
# in the loss variable and gradients in the grads dictionary. Compute
↳ data #
# loss using softmax, and make sure that grads[k] holds the gradients
↳ for #
# self.params[k].
↳ #

↳ #####
loss, dX = softmax_loss(scores, y, L2_REG, W1, W2, lamda=0.01)
dout2, dW2, db2 = fc_backward(dX, cache3, L2_REG, lamda=0.01)
dout1 = relu_backward(dout2, cache2)
dX1, dW1, db1 = fc_backward(dout1, cache1, L2_REG, lamda=0.01)

```

```

        grads['W2'], grads['b2'] = dW2, db2
        grads['W1'], grads['b1'] = dW1, db1

↳ #####
    #                                END OF YOUR CODE                                #
↳ #
    #####
    return loss, grads

```

7 5.2(c) Training

In this problem, you need to preprocess the images and set up model hyperparameters. Notice that adjust the training and val split is optional.

```

[ ]: def unpickle(file):
    with open(file, 'rb') as fo:
        dict = pickle.load(fo, encoding="latin1")
    return dict

def load_cifar10():
    data = {}
    meta = unpickle("cifar-10-batches-py/batches.meta")
    batch1 = unpickle("cifar-10-batches-py/data_batch_1")
    batch2 = unpickle("cifar-10-batches-py/data_batch_2")
    batch3 = unpickle("cifar-10-batches-py/data_batch_3")
    batch4 = unpickle("cifar-10-batches-py/data_batch_4")
    batch5 = unpickle("cifar-10-batches-py/data_batch_5")
    test_batch = unpickle("cifar-10-batches-py/test_batch")
    X_train = np.vstack((batch1['data'], batch2['data'], batch3['data'], \
                          batch4['data'], batch5['data']))
    Y_train = np.array(batch1['labels'] + batch2['labels'] + batch3['labels'] +
                       batch4['labels'] + batch5['labels'])
    X_test = test_batch['data']
    Y_test = test_batch['labels']

    #Preprocess images here
    X_train = (X_train - np.mean(X_train, axis=1, keepdims=True)) / np.
↳ std(X_train, axis=1, keepdims=True)
    X_test = (X_test - np.mean(X_test, axis=1, keepdims=True)) / np.
↳ std(X_test, axis=1, keepdims=True)

    data['X_train'] = X_train[:40000]
    data['y_train'] = Y_train[:40000]
    data['X_val'] = X_train[40000:]
    data['y_val'] = Y_train[40000:]

```

```

data['X_test'] = X_test
data['y_test'] = Y_test
return data

def testNetwork(model, X, y, num_samples=None, batch_size=100, L2_REG=False):
    """
    Check accuracy of the model on the provided data.

    Inputs:
    - model: Image classifier
    - X: Array of data, of shape (N, d_1, ..., d_k)
    - y: Array of labels, of shape (N,)
    - num_samples: If not None, subsample the data and only test the model
      on num_samples datapoints.
    - batch_size: Split X and y into batches of this size to avoid using
      too much memory.

    Returns:
    - acc: Scalar giving the fraction of instances that were correctly
      classified by the model.
    """

    # Subsample the data
    N = X.shape[0]
    if num_samples is not None and N > num_samples:
        mask = np.random.choice(N, num_samples)
        N = num_samples
        X = X[mask]
        y = y[mask]

    # Compute predictions in batches
    num_batches = N // batch_size
    if N % batch_size != 0:
        num_batches += 1
    y_pred = []
    for i in range(num_batches):
        start = i * batch_size
        end = (i + 1) * batch_size
        scores = model.forwards_backwards(X[start:end], L2_REG=L2_REG)
        y_pred.append(np.argmax(scores, axis=1))
    y_pred = np.hstack(y_pred)
    acc = np.mean(y_pred == y)

    return acc

def SGD(W,dW, learning_rate=1e-3):
    """ Apply a gradient descent step on weight W

```

```

Inputs:
    W : Weight matrix
    dW : gradient of weight, same shape as W
    learning_rate : Learning rate. Defaults to 1e-3.
Returns:
    new_W: Updated weight matrix
    """

    # Apply a gradient descent step on weight W using the gradient dW and the
    ↪specified learning rate.
    new_W = W - learning_rate * dW

    return new_W

def trainNetwork(model, data, **kwargs):
    """
    Required arguments:
    - model: Image classifier
    - data: A dictionary of training and validation data containing:
        'X_train': Array, shape (N_train, d_1, ..., d_k) of training images
        'X_val': Array, shape (N_val, d_1, ..., d_k) of validation images
        'y_train': Array, shape (N_train,) of labels for training images
        'y_val': Array, shape (N_val,) of labels for validation images

    Optional arguments:
    - learning_rate: A scalar for initial learning rate.
    - lr_decay: A scalar for learning rate decay; after each epoch the
        learning rate is multiplied by this value.
    - batch_size: Size of minibatches used to compute loss and gradient
        during training.
    - num_epochs: The number of epochs to run for during training.
    - print_every: Integer; training losses will be printed every
        print_every iterations.
    - verbose: Boolean; if set to false then no output will be printed
        during training.
    - num_train_samples: Number of training samples used to check training
        accuracy; default is 1000; set to None to use entire training set.
    - num_val_samples: Number of validation samples to use to check val
        accuracy; default is None, which uses the entire validation set.
    - optimizer: Choice of using either 'SGD' or 'SGD_Momentum' for updating
    ↪weights; default is SGD.
    """

    learning_rate = kwargs.pop('learning_rate', 1e-3)
    lr_decay = kwargs.pop('lr_decay', 1.0)
    batch_size = kwargs.pop('batch_size', 100)

```

```

num_epochs = kwargs.pop('num_epochs', 10)
num_train_samples = kwargs.pop('num_train_samples', 1000)
num_val_samples = kwargs.pop('num_val_samples', None)
print_every = kwargs.pop('print_every', 10)
verbose = kwargs.pop('verbose', True)
optimizer = kwargs.pop('optimizer', 'SGD')

L2_REG = kwargs.pop('L2_REG', False)

epoch = 0
best_val_acc = 0
best_params = {}
loss_history = []
train_acc_history = []
val_acc_history = []

num_train = data['X_train'].shape[0]
iterations_per_epoch = max(num_train // batch_size, 1)
num_iterations = num_epochs * iterations_per_epoch

#Initialize velocity dictionary if optimizer is SGD_Momentum
if optimizer == 'SGD_Momentum':
    velocity_dict = {p:np.zeros(w.shape) for p,w in model.params.items()}

for t in range(num_iterations):
    # Make a minibatch of training data
    batch_mask = np.random.choice(num_train, batch_size)
    X_batch = data['X_train'][batch_mask]
    y_batch = data['y_train'][batch_mask]

    # Compute loss and gradient
    loss, grads = model.forwards_backwards(X_batch, y_batch, L2_REG)
    loss_history.append(loss)

    # Perform a parameter update
    if optimizer == 'SGD':
        for p, w in model.params.items():
            model.params[p] = SGD(w,grads[p], learning_rate=learning_rate)

    elif optimizer == 'SGD_Momentum':
        for p, w in model.params.items():
            model.params[p], velocity_dict[p] = SGD_Momentum(w, grads[p],
↪velocity_dict[p], beta=0.5, learning_rate=learning_rate)
    else:
        raise NotImplementedError
    # Print training loss

```

```

if verbose and t % print_every == 0:
    print('(Iteration %d / %d) loss: %f' % (
        t + 1, num_iterations, loss_history[-1]))

# At the end of every epoch, increment the epoch counter and decay
# the learning rate.
epoch_end = (t + 1) % iterations_per_epoch == 0
if epoch_end:
    epoch += 1
    learning_rate *= lr_decay

# Check train and val accuracy on the first iteration, the last
# iteration, and at the end of each epoch.
first_it = (t == 0)
last_it = (t == num_iterations - 1)
if first_it or last_it or epoch_end:
    train_acc = testNetwork(model, data['X_train'], data['y_train'],
        num_samples= num_train_samples, L2_REG=L2_REG)
    val_acc = testNetwork(model, data['X_val'], data['y_val'],
        num_samples=num_val_samples, L2_REG=L2_REG)
    train_acc_history.append(train_acc)
    val_acc_history.append(val_acc)

    if verbose:
        print('(Epoch %d / %d) train acc: %f; val_acc: %f' % (
            epoch, num_epochs, train_acc, val_acc))

    # Keep track of the best model
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_params = {}
        for k, v in model.params.items():
            best_params[k] = v.copy()

model.params = best_params

return model, train_acc_history, val_acc_history

# load data
data = load_cifar10()
train_data = { k: data[k] for k in ['X_train', 'y_train',
                                    'X_val', 'y_val']}

#####
# TODO: Set up model hyperparameters for SGD #####
#####

```



```

# initialize model
model_SGD = SoftmaxClassifier(hidden_dim = 300, weight_scale=1e-2)

# start training using SGD
model_SGD, train_acc_history_SGD, val_acc_history_SGD = trainNetwork(
    model_SGD, train_data, learning_rate = 0.002,
    lr_decay=0.97, num_epochs=10,
    batch_size=32, print_every=1000, optimizer = 'SGD', L2_REG = False)
#####
#                               END OF YOUR CODE                               #
#####

```

```

(Iteration 1 / 12500) loss: 2.318354
(Epoch 0 / 10) train acc: 0.120000; val_acc: 0.112900
(Iteration 1001 / 12500) loss: 1.911321
(Epoch 1 / 10) train acc: 0.346000; val_acc: 0.338400
(Iteration 2001 / 12500) loss: 1.711310
(Epoch 2 / 10) train acc: 0.393000; val_acc: 0.384300
(Iteration 3001 / 12500) loss: 1.587648
(Epoch 3 / 10) train acc: 0.408000; val_acc: 0.403600
(Iteration 4001 / 12500) loss: 1.437421
(Epoch 4 / 10) train acc: 0.427000; val_acc: 0.419900
(Iteration 5001 / 12500) loss: 1.558689
(Iteration 6001 / 12500) loss: 1.619179
(Epoch 5 / 10) train acc: 0.440000; val_acc: 0.432200
(Iteration 7001 / 12500) loss: 1.671945
(Epoch 6 / 10) train acc: 0.463000; val_acc: 0.443900
(Iteration 8001 / 12500) loss: 1.358775
(Epoch 7 / 10) train acc: 0.463000; val_acc: 0.450000
(Iteration 9001 / 12500) loss: 1.750838
(Epoch 8 / 10) train acc: 0.526000; val_acc: 0.458500
(Iteration 10001 / 12500) loss: 1.522574
(Iteration 11001 / 12500) loss: 1.451228
(Epoch 9 / 10) train acc: 0.542000; val_acc: 0.465800
(Iteration 12001 / 12500) loss: 1.313443
(Epoch 10 / 10) train acc: 0.545000; val_acc: 0.472000

```

8 5.2(d) Training with SGD_Momentum

The model above was trained using SGD. Now implement the SGD_Momentum function to train the model using SGD with momentum.

```

[ ]: def SGD_Momentum(W, dW, velocity, beta=0.5, learning_rate=1e-3):
    """ Apply a gradient descent with momentum update on weight W
    Inputs:
        W : Weight matrix
        dW : gradient of weight, same shape as W

```

```

        velocity : velocity matrix, same shape as W
        beta : scalar value in range [0,1] weighting the velocity matrix.
    ↪Setting it to 0 should make SGD_Momentum same as SGD.
        Defaults to 0.5.
        learning_rate : Learning rate. Defaults to 1e-3.
    Returns:
        new_W: Updated weight matrix
        new_velocity: Updated velocity matrix
    """
    # ===== your code here! =====
    # TODO:
    # Apply a gradient descent step on weight W using the gradient dW and the
    ↪specified learning rate.
    # 1. Calculate the new velocity by using the velocity of last iteration
    ↪(input velocity) and gradient
    # 2. Update the weights using the new_velocity
    new_velocity = beta * velocity + learning_rate * dW
    new_W = W - new_velocity
    # ===== end of code =====
    return new_W, new_velocity

#####
# TODO: Set up model hyperparameters for SGD_Momentum
# Your hyperparameters should be identical to what you used for SGD (without
    ↪momentum)#
#####

# initialize model
model_SGD_Momentum = SoftmaxClassifier(hidden_dim = 300, weight_scale=1e-2)

# start training
#Using SGD_Momentum as optimizer for training for training
model_SGD_Momentum, train_acc_history_SGD_Momentum,
    ↪val_acc_history_SGD_Momentum = trainNetwork(
        model_SGD_Momentum, train_data, learning_rate = 0.002,
        lr_decay=0.97, num_epochs=10,
        batch_size=32, print_every=1000, optimizer = 'SGD_Momentum', L2_REG = False)
#####
#                               END OF YOUR CODE                               #
#####

```

```

(Iteration 1 / 12500) loss: 2.296431
(Epoch 0 / 10) train acc: 0.083000; val_acc: 0.091500
(Iteration 1001 / 12500) loss: 1.660196
(Epoch 1 / 10) train acc: 0.388000; val_acc: 0.374600
(Iteration 2001 / 12500) loss: 1.516907
(Epoch 2 / 10) train acc: 0.458000; val_acc: 0.421700

```

```
(Iteration 3001 / 12500) loss: 1.415678
(Epoch 3 / 10) train acc: 0.492000; val_acc: 0.442400
(Iteration 4001 / 12500) loss: 1.427885
(Epoch 4 / 10) train acc: 0.489000; val_acc: 0.462300
(Iteration 5001 / 12500) loss: 1.284780
(Iteration 6001 / 12500) loss: 1.662088
(Epoch 5 / 10) train acc: 0.503000; val_acc: 0.470100
(Iteration 7001 / 12500) loss: 1.564698
(Epoch 6 / 10) train acc: 0.525000; val_acc: 0.480400
(Iteration 8001 / 12500) loss: 1.652982
(Epoch 7 / 10) train acc: 0.570000; val_acc: 0.485100
(Iteration 9001 / 12500) loss: 1.245100
(Epoch 8 / 10) train acc: 0.552000; val_acc: 0.494400
(Iteration 10001 / 12500) loss: 1.120618
(Iteration 11001 / 12500) loss: 1.256265
(Epoch 9 / 10) train acc: 0.569000; val_acc: 0.499600
(Iteration 12001 / 12500) loss: 1.207839
(Epoch 10 / 10) train acc: 0.580000; val_acc: 0.498000
```

9 5.2(e) Report Accuracy

Run the given code and report the accuracy of model_SGD and model_SGD_Momentum on test set. Which model trains more quickly? Is the ultimate validation accuracy different? Report your observation in the text block below.

```
[ ]: # report test accuracy
acc = testNetwork(model_SGD, data['X_test'], data['y_test'])
print("Test accuracy of model_SGD: {}".format(acc))
# report test accuracy
acc = testNetwork(model_SGD_Momentum, data['X_test'], data['y_test'])
print("Test accuracy of model_SGD_Momentum: {}".format(acc))
```

Test accuracy of model_SGD: 0.48

Test accuracy of model_SGD_Momentum: 0.4953

My observation: The model trains faster with SGD Momentum to reach similar training accuracy, indicating that with momentum, it helps minimizing loss function faster. While the validation accuracy of SDG Momentum is similar to that of SGD, only slightly higher. # 5.2(f) Plot

Using the train_acc_history and val_acc_history, plot the train & val accuracy versus epochs on one plot, using SGD and SGD_Momentum as optimizer.

```
[ ]: #####
# Your Code here
#####
def plot_acc_history(train_acc_history_SGD, val_acc_history_SGD,
    ↪train_acc_history_SGD_Momentum, val_acc_history_SGD_Momentum, L2_REG=False):
    # Expect these variables to exist after training:
    # Convert histories to numpy arrays for easy slicing
```

```

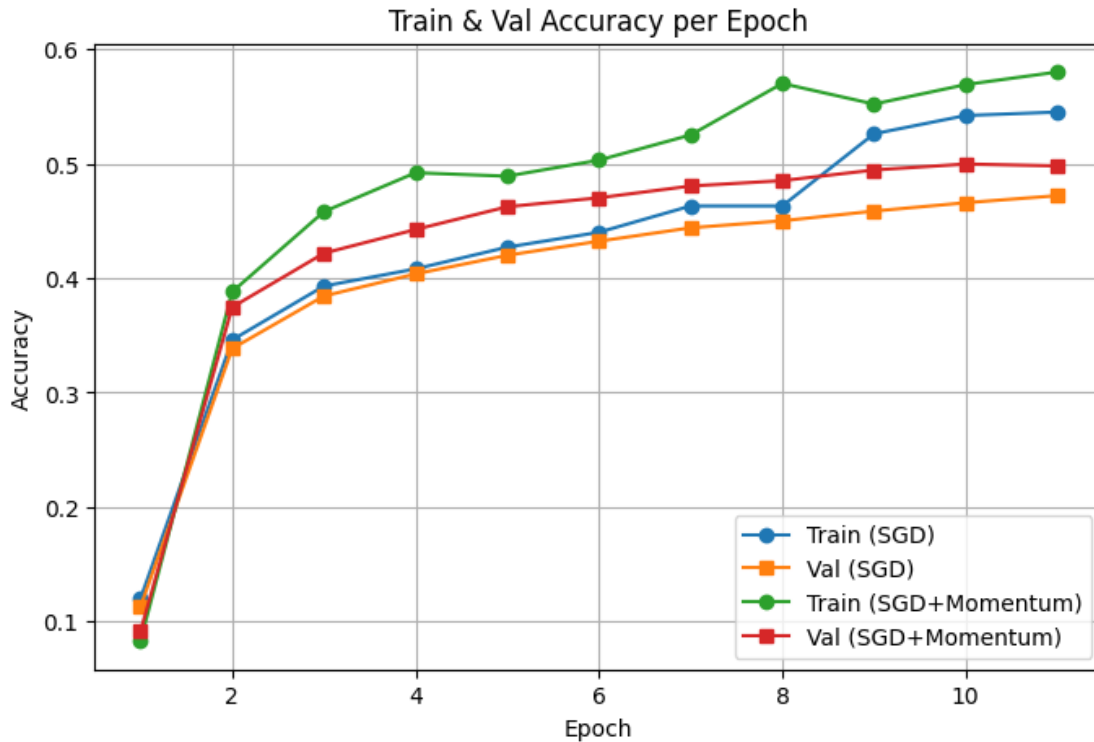
train_s = np.array(train_acc_history_SGD)
val_s = np.array(val_acc_history_SGD)
train_sm = np.array(train_acc_history_SGD_Momentum)
val_sm = np.array(val_acc_history_SGD_Momentum)

# Histories were recorded once per epoch (see trainNetwork), so x-axis
↳ is epochs
epochs_s = np.arange(1, len(train_s) + 1)
epochs_sm = np.arange(1, len(train_sm) + 1)

plt.figure(figsize=(8,5))
plt.plot(epochs_s, train_s, '-o', label='Train (SGD)')
plt.plot(epochs_s, val_s, '-s', label='Val (SGD)')
plt.plot(epochs_sm, train_sm, '-o', label='Train (SGD+Momentum)')
plt.plot(epochs_sm, val_sm, '-s', label='Val (SGD+Momentum)')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
if L2_REG:
    plt.title('Train & Val Accuracy per Epoch (with L2
↳ regularization)')
else:
    plt.title('Train & Val Accuracy per Epoch')
plt.legend()
plt.grid(True)
plt.show()

# Plot the training and validation accuracy history for SGD and SGD_Momentum
↳ without L2 regularization
plot_acc_history(train_acc_history_SGD, val_acc_history_SGD,
↳ train_acc_history_SGD_Momentum, val_acc_history_SGD_Momentum, L2_REG=False)
#####
#                               END OF YOUR CODE                               #
#####

```



9.1 (Optional) Add L2 Regularization to model and perform model training

```
[ ]: # initialize model
model_SGD = SoftmaxClassifier(hidden_dim = 300, weight_scale=1e-2)

# start training using SGD
model_SGD_L2, train_acc_history_SGD_L2, val_acc_history_SGD_L2 = trainNetwork(
    model_SGD, train_data, learning_rate = 0.002,
    lr_decay=0.97, num_epochs=10,
    batch_size=32, print_every=1000, optimizer = 'SGD', L2_REG=True)

# initialize model
model_SGD_Momentum = SoftmaxClassifier(hidden_dim = 300, weight_scale=1e-2)

# start training
#Using SGD_Momentum as optimizer for training for training
model_SGD_Momentum_L2, train_acc_history_SGD_Momentum_L2,
    val_acc_history_SGD_Momentum_L2 = trainNetwork(
    model_SGD_Momentum, train_data, learning_rate = 0.002,
    lr_decay=0.97, num_epochs=10,
    batch_size=32, print_every=1000, optimizer = 'SGD_Momentum', L2_REG=True)
```

```

# report test accuracy
acc = testNetwork(model_SGD_L2, data['X_test'], data['y_test'], L2_REG=True)
print("Test accuracy of model_SGD_L2: {}".format(acc))
# report test accuracy
acc = testNetwork(model_SGD_Momentum_L2, data['X_test'], data['y_test'],
    ↪L2_REG=True)
print("Test accuracy of model_SGD_Momentum_L2: {}".format(acc))

# Plot the training and validation accuracy history for SGD and SGD_Momentum
    ↪with L2 regularization
plot_acc_history(train_acc_history_SGD_L2, val_acc_history_SGD_L2,
    ↪train_acc_history_SGD_Momentum_L2, val_acc_history_SGD_Momentum_L2,
    ↪L2_REG=True)

# Plot the accuracy deviation history for SGD and SGD_Momentum with L2
    ↪regularization
def plot_acc_dev_history(train_acc_history_SGD, val_acc_history_SGD,
    ↪train_acc_history_SGD_Momentum, val_acc_history_SGD_Momentum,
    ↪train_acc_history_SGD_L2, val_acc_history_SGD_L2,
    ↪train_acc_history_SGD_Momentum_L2, val_acc_history_SGD_Momentum_L2):
    # Expect these variables to exist after training:
    # Convert histories to numpy arrays for easy slicing
    train_s = np.array(train_acc_history_SGD)
    val_s = np.array(val_acc_history_SGD)
    train_sm = np.array(train_acc_history_SGD_Momentum)
    val_sm = np.array(val_acc_history_SGD_Momentum)

    train_s_L2 = np.array(train_acc_history_SGD_L2)
    val_s_L2 = np.array(val_acc_history_SGD_L2)
    train_sm_L2 = np.array(train_acc_history_SGD_Momentum_L2)
    val_sm_L2 = np.array(val_acc_history_SGD_Momentum_L2)

    epochs_s = np.arange(1, len(train_s) + 1)
    epochs_sm = np.arange(1, len(train_sm) + 1)

    plt.figure(figsize=(8,5))
    plt.plot(epochs_s, abs(train_s - val_s), '-o', label='Accuracy
    ↪Deviation (SGD)')
    plt.plot(epochs_s, abs(train_s_L2 - val_s_L2), '-s', label='Accuracy
    ↪Deviation (SGD+L2)')
    plt.plot(epochs_sm, abs(train_sm - val_sm), '-o', label='Accuracy
    ↪Deviation (SGD+Momentum)')
    plt.plot(epochs_sm, abs(train_sm_L2 - val_sm_L2), '-s', label='Accuracy
    ↪Deviation (SGD+Momentum+L2)')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy Deviation')

```

```

plt.title('Accuracy Deviation Between Train & Val per Epoch')
plt.legend()
plt.grid(True)
plt.show()

plot_acc_dev_history(train_acc_history_SGD, val_acc_history_SGD,
    ↳train_acc_history_SGD_Momentum, val_acc_history_SGD_Momentum,
    ↳train_acc_history_SGD_L2, val_acc_history_SGD_L2,
    ↳train_acc_history_SGD_Momentum_L2, val_acc_history_SGD_Momentum_L2)

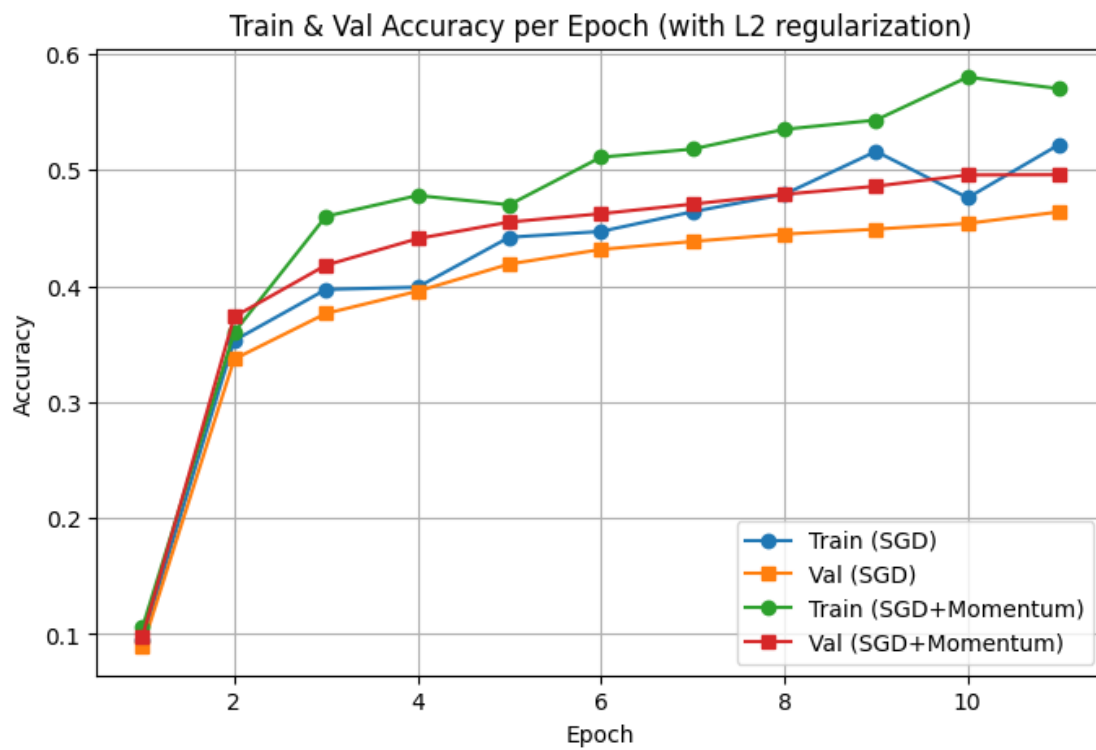
```

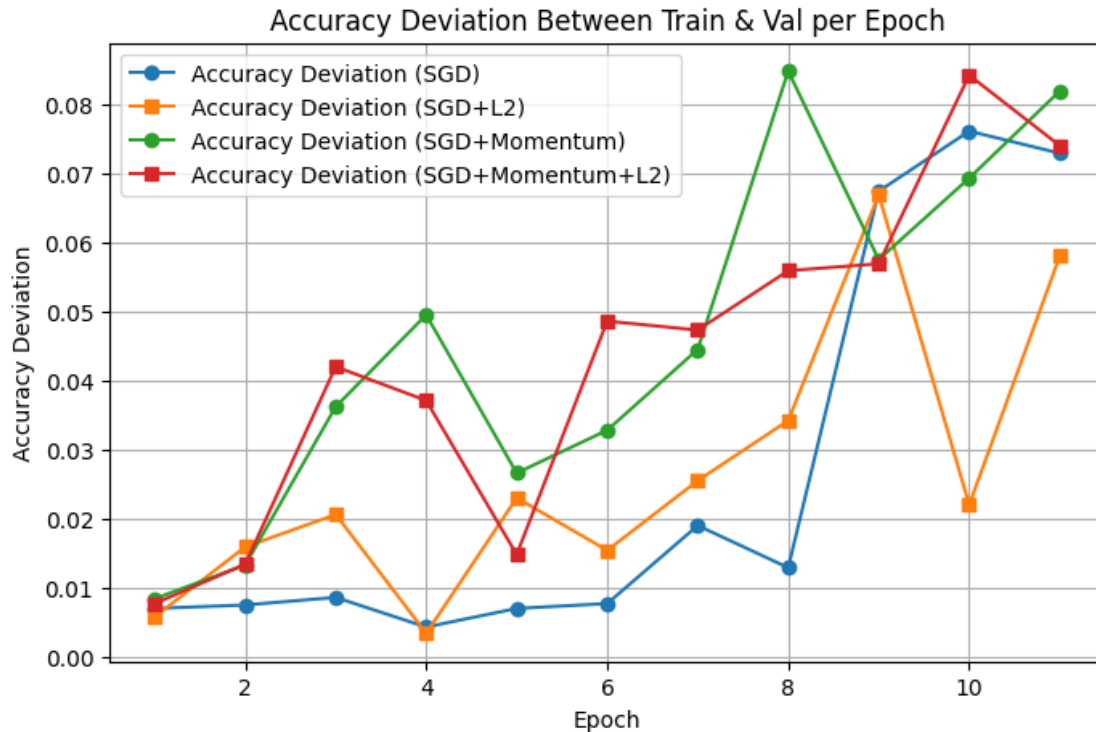
```

(Iteration 1 / 12500) loss: 3.248711
(Epoch 0 / 10) train acc: 0.095000; val_acc: 0.089100
(Iteration 1001 / 12500) loss: 2.778225
(Epoch 1 / 10) train acc: 0.353000; val_acc: 0.337000
(Iteration 2001 / 12500) loss: 2.719791
(Epoch 2 / 10) train acc: 0.397000; val_acc: 0.376300
(Iteration 3001 / 12500) loss: 2.467616
(Epoch 3 / 10) train acc: 0.399000; val_acc: 0.395500
(Iteration 4001 / 12500) loss: 2.277315
(Epoch 4 / 10) train acc: 0.442000; val_acc: 0.418900
(Iteration 5001 / 12500) loss: 2.491720
(Iteration 6001 / 12500) loss: 2.470770
(Epoch 5 / 10) train acc: 0.447000; val_acc: 0.431500
(Iteration 7001 / 12500) loss: 2.518345
(Epoch 6 / 10) train acc: 0.464000; val_acc: 0.438400
(Iteration 8001 / 12500) loss: 2.553082
(Epoch 7 / 10) train acc: 0.479000; val_acc: 0.444700
(Iteration 9001 / 12500) loss: 2.131522
(Epoch 8 / 10) train acc: 0.516000; val_acc: 0.449000
(Iteration 10001 / 12500) loss: 1.969747
(Iteration 11001 / 12500) loss: 2.104688
(Epoch 9 / 10) train acc: 0.476000; val_acc: 0.453900
(Iteration 12001 / 12500) loss: 2.052464
(Epoch 10 / 10) train acc: 0.522000; val_acc: 0.463900
(Iteration 1 / 12500) loss: 3.215283
(Epoch 0 / 10) train acc: 0.106000; val_acc: 0.098200
(Iteration 1001 / 12500) loss: 2.533199
(Epoch 1 / 10) train acc: 0.360000; val_acc: 0.373500
(Iteration 2001 / 12500) loss: 2.431280
(Epoch 2 / 10) train acc: 0.460000; val_acc: 0.417900
(Iteration 3001 / 12500) loss: 2.243614
(Epoch 3 / 10) train acc: 0.478000; val_acc: 0.440800
(Iteration 4001 / 12500) loss: 2.543602
(Epoch 4 / 10) train acc: 0.470000; val_acc: 0.455100
(Iteration 5001 / 12500) loss: 1.997473
(Iteration 6001 / 12500) loss: 1.929818
(Epoch 5 / 10) train acc: 0.511000; val_acc: 0.462300

```

(Iteration 7001 / 12500) loss: 2.121202
(Epoch 6 / 10) train acc: 0.518000; val_acc: 0.470600
(Iteration 8001 / 12500) loss: 2.154812
(Epoch 7 / 10) train acc: 0.535000; val_acc: 0.479000
(Iteration 9001 / 12500) loss: 1.997990
(Epoch 8 / 10) train acc: 0.543000; val_acc: 0.486000
(Iteration 10001 / 12500) loss: 1.740097
(Iteration 11001 / 12500) loss: 1.811902
(Epoch 9 / 10) train acc: 0.580000; val_acc: 0.495700
(Iteration 12001 / 12500) loss: 1.912959
(Epoch 10 / 10) train acc: 0.570000; val_acc: 0.495900
Test accuracy of model_SGD_L2: 0.4726
Test accuracy of model_SGD_Momentum_L2: 0.5007





10 Convert Notebook to PDF

```
[ ]: # generate pdf
# Please provide the full path of the notebook file below
# Important: make sure that your file name does not contain spaces!
import os
notebookpath = '/content/drive/My Drive/EECS 442 - Computer Vision/Hw/hw5/
↳ pohsun_40183452.ipynb' # Ex: notebookpath = '/content/drive/My Drive/Colab_
↳ Notebooks/EECS 442 Fall 2023 - PS1.ipynb'
drive_mount_point = '/content/drive/'
from google.colab import drive
drive.mount(drive_mount_point)
file_name = notebookpath.split('/')[-1]
get_ipython().system("apt update && apt install texlive-xetex_
↳ texlive-fonts-recommended texlive-generic-recommended")
get_ipython().system("pip install py pandoc")
get_ipython().system("apt-get install texlive texlive-xetex texlive-latex-extra_
↳ pandoc")
get_ipython().system("jupyter nbconvert --to PDF {}".format(notebookpath.
↳ replace(' ', '\\ ')))
from google.colab import files
files.download(notebookpath.split('.')[0]+'pdf')
```